CS339: Abstractions and Paradigms for Programming

Midsem Exam (2 hours; 30 marks)

September 17^{th}, 2024

Instructions:

- Write neat, clear and crisp answers in the space provided.
- There are separate spaces provided for rough work.
- In case you make an assumption, write it down before the corresponding answer.
- Each **Q** carries 6 marks.

Q1. Consider the following bank-account procedure.

```
(define (make-account balance)
  (define (withdraw amount)
    (if (>= balance amount)
        (begin (set! balance (- balance amount))
               balance)
        "Insufficient funds"))
  (define (dispatch m)
    (cond ((eq? m 'withdraw) withdraw)
          (else
            (error "Unknown request: MAKE-ACCOUNT" m))))
  dispatch)
```

(a) Draw the complete environment structure after executing the following sequence of interactions: [3]

```
> (define acc (make-account 50))
> ((acc 'withdraw) 30)
20
```

(b) Where is the local state for `acc` kept? Indicate by numbering the boxes in part (a) and just write the right number as your answer. [1]

(c) Suppose we define another account:

```
(define acc2 (make-account 100))
```

How are the local states for the two accounts kept distinct? Which parts of the environment structure can be shared between `acc` and `acc2`? [2]

SPACE FOR YOUR MUSINGS

Q2. (a) What would the following code result under (i) static scoping; and (ii) dynamic scoping. [2]

```
(define x 20)
(define (foo g)
  (define x 30)
  (lambda (y)
    (+ x y g)))
(define bar (foo 15))
(bar 30)
```

(b) In a similar vein as we had distinguished static and dynamic resolution for polymorphism, explain why static scoping is *static* and dynamic scoping, *dynamic*? [2]

(c) What would you change and how, in your answer to 1(a), if Scheme had dynamic scoping? [2]

Q3. State true or false and explain your answer using a code example.

(a) The special forms `define` and `set!` do the same thing. [2]

(b) Lists would become streams under normal-order evaluation. [2]

(c) Substitution model does not work in presence of assignment statements. [2]

Q4. (a) Derive the normal form of the following expressions. Do not skip any β-reduction step.

(i) $(\lambda x.x)\ (\lambda y.yy)\ (\lambda z.za)$ [1]

(ii) $(\lambda x.\lambda y.xyy)\ (\lambda y.y)\ y$ [1]

(iii) $not\ (not\ true)$, where $not = \lambda x.((x\ false)\ true)$; $true = \lambda x.\lambda y.x$; and $false = \lambda x.\lambda y.y$ [2]

(b) Match the columns by drawing clear straight lines. [2]

Eager evaluation	Polymorphism
Message passing	Call by value
Has-a relationship	Thunk
Lazy evaluation	Composition

Q5. Peppy Paneer tried to define the stream constructor `stream-cons` and the stream selectors `stream-car` and `stream-cdr` as follows:

```
(define (cons-stream a b) (cons a (delay b))  ; 1
(define (stream-car s) (car s))               ; 2
(define (stream-cdr s) (force (cdr s)))       ; 3
(define (delay e) (lambda () e))              ; 4
(define (force p) (p))                        ; 5
```

(a) Write numbers corresponding to the definitions that will not work. [1]

(b) What is the exact problem with the ones that do not work? Suggest two different approaches in which one could handle the problem. [2]

(c) Betty Butter wrote the following functional expression to increment the squares of the numbers present in a given list `l`: `(map inc (map square l))`; assume `inc` and `square` increment and square their arguments, respectively, and that `map` is defined as usual. On the other hand, Cheesy Corn claims to have another functional version that is more memory-efficient.

(i) Guess what the Cheesy version is. [1]

(ii) Take the list `(1 2 3 4)` and explain how Cheesy's version is better. Can you come up with yet another functional solution different from the Cheesy version that still is memory efficient?[1]. [2]

PCQ. Upload a meme about your favorite moment(s) till now in this course on Moodle by the end of Friday. Up to two PCs on offer based on how much we like it. (All of them would be made public later.)

[1] If you started thinking about the von Neumann architecture from Activity 1 after seeing this question, do talk to me some time!

SPACE FOR YOUR MUSINGS

SPACE FOR YOUR MUSINGS